T4 - SQLite with Python (`sqlite3`)

Covers: connect / cursor / CREATE TABLE / INSERT / SELECT / UPDATE / DELETE, "no duplicates", year filtering with `strftime`, DataFrame <-> database, and the exact 2024 Tasks 2a + 2b. Earns marks in: 2024 Task 2 (15 marks), 2023 Task 1a/1b (ISBN check), 2022 (TravelPrice), 2021 (Users + duplicate usernames).

1. What the exam asks

2024 paper, Python Task 2: Python Database Management [15 marks] - "*create a database to store and manage the sales data processed in Part 1.*"

Sub-task	Wording	Marks
2a	Create an SQLite database named **SalesDB**	2
2a	Create a table named **Sales** with appropriate columns (**Date, Product, Quantity, Price, Total**)	3
2a	Insert the cleaned data from Part 1 into Sales. **Ensure no duplicate entries are added.**	4
2b	Total Sales Calculation: query the **total sales for the year 2023** and display it	3
2b	Product Sales Summary: list each product and its **total quantity sold in 2023**, **descending order of quantity**	3

Older papers (same patterns, shown here WITHOUT tkinter): - 2021 Task 1: database `Users`; save username + password; "*should not save duplicated usernames ... check if the username already exists*" [5]; password >= 8 characters AND contains a number [6]; "Clear" and "Display". - 2022 Task 1b: database `TravelPrice`; save each passenger's age and ticket price; display ages, display prices, display total cost [5]; "*Clear All*" *deletes everything from the database* [2]. - 2023 Task 1a/1b: database `LibraryDB`; save ISBN/title/author/year; *no duplicate ISBNs*; year between 1450 and now; title and author not empty; then answer "Recommend me a book by [author]" and "When was [title] published" from the database. - Lab 3 Exercises 17/18 (PhoneBook1, table Names: view / add / search surname / delete by ID); tAPP-3 Problem 1 (`employees` database, `contactDetails` table).

The data file (`sales_data.csv`, 151 rows) is deliberately messy:

Problem	Where	Effect
Dates are `dd/mm/yyyy`	whole file	need `dayfirst=True`, then convert to `YYYY-MM-DD` text for SQLite
Years **2022, 2023, 2024** (29 / 100 / 22 rows)	whole file	the year filter really matters
Blank Product (6 rows), blank Price (1), blank Total (3)	e.g. rows for 24/02/2023, 12/03/2023	clean *before* inserting
The week 04/04/2022 to 10/04/2022 appears **twice**	rows 123-129 and 145-151	6 exact duplicate rows (the 7th differs only by a blank Product) - the "no duplicates" rule is really tested

ADDED - The course instructor's official checklist (practical assessment) - "*SQLite with Python*", quoted word for word: - creating and connecting to an SQLite database - storing a Pandas DataFrame in a database table - writing SELECT, WHERE, ORDER BY and UPDATE queries - using parameterised queries with ? - committing database changes - retrieving query results and exporting them to CSV

The instructor adds: "*Do not focus on memorising one complete solution. Instead, practise the individual operations and understand what each line of code is doing.*" Each bullet has its own small, independent section in **Section 8** (8.1 to 8.7); 8.8 is a cheat-sheet of the 10 SQL statements.

Checklist bullet	Where in this document
creating and connecting	3.1 and **8.1**
storing a DataFrame in a table	3.9, 3.10 and **8.2**
SELECT, WHERE, ORDER BY	3.5 to 3.7 and **8.3**

Mistake	What you see	Fix
Comparing year to a number: `strftime(...) = 2023`	No rows (text vs number)	Quote it: `= '2023'`.
ADDED `UPDATE` with no `WHERE`	No error: `cursor.rowcount` equals the number of ALL rows (5 of 5) and every row changes	Always write `WHERE`; print `cursor.rowcount` and compare with what you expect; `conn.rollback()` undoes it if not yet committed.
ADDED Trusting `rowcount` after a SELECT	`cursor.rowcount` is `-1` after a SELECT	`rowcount` is only meaningful after INSERT/UPDATE/DELETE. Use `len(cursor.fetchall())` for SELECT.
ADDED UPDATE matched nothing (misspelt name)	No error, `rowcount` is `0`, nothing changes	Check `rowcount == 0` and print "No such record".
ADDED Changing `Price` but not `Total`	Stale `Total` column (50.0 next to Price 11.0)	`SET Price = ?, Total = Quantity * ?` in one UPDATE.
ADDED NumPy number as a `?` value in a WHERE	**No error, no rows** (`[]`): it is compared as a BLOB	`int(x)` / `float(x)` before passing it.
ADDED `LIKE '%?%'`	`ProgrammingError: Incorrect number of bindings supplied. The current statement uses 0, and there are 1 supplied.` (`?` inside quotes is just a character)	Put the `%` in the value: `LIKE ?` with `(f"%{word}%",)`.
ADDED `SELECT * FROM ?` (table name as parameter)	`OperationalError: near "?": syntax error`	`?` only works for **values**, never for table or column names.
ADDED Running the script from another folder	New empty `cheat.db` is created there, then `OperationalError: no such table: Sales`	`connect("x.db")` is relative to the **working folder**; run from the script's folder or use `os.path.exists` to check.
ADDED `with conn:` expecting it to close	Later `conn.execute` still works; the file stays open	`with conn:` only commits (or rolls back on error). Call `conn.close()` yourself.
ADDED `df.to_csv("out.csv")` without `index=False`	First column is an unnamed `0,1,2,...` index (`,Product,TotalQty,Revenue`)	`df.to_csv("out.csv", index=False)`.
ADDED Exporting with `SUM(Total)` and no alias	Header in the CSV is literally `SUM(Total)`	Name columns: `SUM(Total) AS Revenue`.

6. Anticipated exam questions

Q1 [2 marks] *Create an SQLite database named SalesDB and a connection/cursor.*

```
conn = sqlite3.connect("SalesDB.db")
cursor = conn.cursor()
```

Q2 [3 marks] *Create a table Sales with columns Date, Product, Quantity, Price, Total.* Template 3.1 (TEXT, TEXT, INTEGER, REAL, REAL; add `UNIQUE (Date, Product)`).

Q3 [4 marks] *Insert the cleaned DataFrame into Sales; ensure no duplicate entries are added.* Template 3.9 with the table from 3.1 and a one-line comment: `# UNIQUE(Date, Product) + OR IGNORE prevent duplicates`.

Q4 [3 marks] *Display the total sales for 2023.*

```
cursor.execute("""SELECT SUM(Total) FROM Sales
                  WHERE strftime('%Y', Date) = '2023'""")
print("Total sales 2023:", cursor.fetchone()[0])
```

Q5 [3 marks] *List each product and its total quantity sold in 2023, highest first.* Template 3.7.

Q6 [3 marks] *Why does* `INSERT OR IGNORE` *not stop duplicates in this table?* (a table without UNIQUE/PRIMARY KEY) Because "ignore" only applies when a constraint is violated; with no constraint, every insert succeeds. Fix: add `UNIQUE (Date, Product)`.

Q7 [3 marks] *Before saving a new user, check whether the username already exists and print a message if it does.* Template 3.4 + `if username_exists(...): print("Username already exists!")`.

Q8 [2 marks] *Delete everything from the database (Clear All).* `cursor.execute("DELETE FROM Tickets")` then `conn.commit()`.

Q9 [3 marks] *Read all rows of Sales into a DataFrame and print the average Price per product.*

```
df2 = pd.read_sql_query("SELECT * FROM Sales", conn)
print(df2.groupby("Product")["Price"].mean())
```

(or in SQL: `SELECT Product, AVG(Price) FROM Sales GROUP BY Product`).

Q10 [3 marks] *Why must dates be stored as* `YYYY-MM-DD` *text?* SQLite has no date type; its `strftime`/`date` functions only understand ISO text, and it sorts correctly alphabetically. `dd/mm/yyyy` gives `None` from `strftime`.

ADDED - questions in the style of the instructor's checklist

Q11 [3 marks] *Store the DataFrame* `df` *in a table called Sales in* `SalesDB.db`*, replacing any older table, and show it was stored.*

```
df.to_sql("Sales", conn, if_exists="replace", index=False)
print(pd.read_sql_query("SELECT COUNT(*) AS n FROM Sales", conn))
print(df.shape)                  # the row count must match
```

Q12 [3 marks] *Increase the Price of every Chocolate sale by 10% and report how many rows changed.*

```
cursor.execute("UPDATE Sales SET Price = Price * 1.1 WHERE Product = ?",
               ("Chocolate",))
conn.commit()
print(cursor.rowcount, "row(s) updated")
```

Q13 [3 marks] *Show the 3 most expensive sales of 2023 (Product and Total only), biggest first.*

```
cursor.execute("""SELECT Product, Total FROM Sales
                  WHERE strftime('%Y', Date) = ?
                  ORDER BY Total DESC LIMIT 3""", ("2023",))
print(cursor.fetchall())
```

Q14 [4 marks] *Ask the user for a product name and display its sales, using a parameterised query. Why not use an f-string?*

```
name = input("Product: ")
cursor.execute("SELECT * FROM Sales WHERE Product = ?", (name,))
for row in cursor.fetchall():
    print(row)
```

An f-string puts user text *inside* the SQL: a quote breaks it (`O'Brien`) and `x' OR '1'='1` returns every row (SQL injection). `?` keeps the data separate from the command.

Q15 [4 marks] *Export the 2023 product summary (Product, TotalQty) to* `summary_2023.csv` *and read the file back to check it.*

```
result = pd.read_sql_query("""SELECT Product, SUM(Quantity) AS TotalQty
        FROM Sales WHERE strftime('%Y', Date) = ?
        GROUP BY Product ORDER BY TotalQty DESC""", conn, params=("2023",))
result.to_csv("summary_2023.csv", index=False)
print(pd.read_csv("summary_2023.csv"))
```

Q16 [2 marks] *What happens to an INSERT if the program ends before* `conn.commit()`*? What does* `with conn:` *do?* The change is rolled back / lost (other connections never saw it). `with conn:` commits when the block ends normally and rolls back if an error occurs; it does **not** close the connection.

---

7. Quick-reference box (copy onto your notes sheet)

```
import sqlite3            conn = sqlite3.connect("X.db"); cur = conn.cursor()
cur.execute("""CREATE TABLE IF NOT EXISTS T (id INTEGER PRIMARY KEY
   AUTOINCREMENT, name TEXT NOT NULL, d TEXT, q INTEGER, p REAL,
   UNIQUE (d, name))""")      types: INTEGER REAL TEXT BLOB
INSERT: cur.execute("INSERT INTO T (a,b) VALUES (?,?)", (x, y)); conn.commit()
NO DUPES: needs UNIQUE/PRIMARY KEY in table + INSERT OR IGNORE (rowcount 0/1)
          (OR IGNORE alone, no constraint = NOTHING prevented!) or check first
          SELECT 1 FROM T WHERE k=? -> cur.fetchone() is not None
SELECT: SELECT cols FROM T WHERE c=? GROUP BY g ORDER BY x DESC
        fetchone() -> tuple/None | fetchall() -> list of tuples
AGG: COUNT(*) SUM(c) AVG(c) MIN(c) MAX(c)  (SUM of no rows = None)
YEAR: WHERE strftime('%Y', Date) = '2023'   (Date must be YYYY-MM-DD text)
UPDATE T SET c=? WHERE k=?    DELETE FROM T WHERE k=?    DELETE FROM T = clear
ALWAYS conn.commit() after changes; conn.close() at the end
Single param needs comma: (x,)   cast numpy: int(x) float(x)   no f-string SQL
DF -> table: loop df.iterrows() + INSERT OR IGNORE ... int()/float()
             or df.to_sql("T", conn, if_exists="append", index=False)
Table -> DF: pd.read_sql_query("SELECT * FROM T", conn)
Dates: df["Date"].dt.strftime("%Y-%m-%d") BEFORE inserting
```

ADDED to the quick-reference box (instructor checklist items)

```
CONNECT: sqlite3.connect("x.db") = file in the WORKING FOLDER; ":memory:"
   list tables: SELECT name FROM sqlite_master WHERE type='table'
   columns:     PRAGMA table_info(T)   -> (cid, name, type, ...)
DF->TABLE: df.to_sql("T", conn, if_exists="fail"|"replace"|"append",
           index=False, dtype={"c": "TEXT"})   fail = error if table exists
   int->INTEGER float->REAL str->TEXT bool->0/1 NaN->NULL; dates -> strftime
   check: pd.read_sql_query("SELECT * FROM T", conn); compare df.shape
WHERE: = != < > <= >= | AND OR NOT (use brackets) | IN (a,b) | BETWEEN a AND b
   LIKE 'A%' starts | '%x%' contains  (% inside the VALUE: (f"%{w}%",))
ORDER BY a ASC, b DESC | LIMIT n | DISTINCT | col AS alias
UPDATE T SET a=?, b=a*1.1 WHERE k=?  <- check cur.rowcount; conn.commit()
   rowcount: rows changed (0 = nothing matched; -1 after SELECT)
PARAMS: one item = (x,) | IN list: ", ".join("?" * n) | never f-string
   ? is for VALUES only (not table/column names); numpy -> int()/float()
COMMIT: no commit = lost | with conn: commits, rolls back on error, NOT close
EXPORT: pd.read_sql_query(sql, conn, params=(x,)).to_csv("o.csv", index=False)
   or rows=cur.fetchall(); hdr=[d[0] for d in cur.description];
      csv.writer: writerow(hdr); writerows(rows)   (open "w", newline="")
   or pd.DataFrame(rows, columns=hdr).to_csv("o.csv", index=False)
   verify: pd.read_csv("o.csv")
```

---

8. ADDED - The instructor's checklist as INDEPENDENT operations

Each operation below stands alone: copy just the one you need. Every operation has **a template, what every part does, an analogy, and a verify step** (a line that *proves* it worked - examiners like to see that). All code in 8.1 to 8.8 was run and tested.

8.1 Create and connect to an SQLite database

Analogy: walking up to the **filing cabinet** and opening it. If the cabinet does not exist yet, a new empty one is built for you.

WHEN: the first lines of every database program.

```
import os.path
import sqlite3

conn = sqlite3.connect("shop.db")        # open (or create) the cabinet
cursor = conn.cursor()                   # hire the clerk
```

What every part does: - `import sqlite3` - the library is built into Python; nothing to install. - `sqlite3.connect("shop.db")` - returns a **connection**: the open file. If `shop.db` is missing it is created (an empty file) in the *working folder*. `":memory:"` instead of a file name gives a throw-away database that vanishes when the program ends (good for testing). - `conn.cursor()` - returns the **cursor** (the clerk) that runs SQL with `cursor.execute(...)`.

VERIFY:

```
print(os.path.exists("shop.db"))         # True: the file is on disk
cursor.execute("SELECT name FROM sqlite_master WHERE type = 'table'")
print(cursor.fetchall())                 # [] new/empty;  [('Sales',)] later
```

`sqlite_master` is the cabinet's own index card listing every table. Close with `conn.close()` when finished.

8.2 Store a pandas DataFrame in a table (`df.to_sql`)

Analogy: **photocopying a whole spreadsheet into a drawer** in one go: pandas builds the drawer (CREATE TABLE) and files every row (INSERT) for you.

WHEN: "store the DataFrame in the database" and you do not need a UNIQUE rule (if you do, use 3.9/3.10).

```
df["Date"] = df["Date"].dt.strftime("%Y-%m-%d")      # dates -> TEXT first
df.to_sql("Sales", conn, if_exists="fail", index=False)
```

What every part does:

Part	Meaning
`"Sales"`	the table name to create (or add to)
`conn`	the open sqlite3 connection (8.1)
`if_exists="fail"`	**default**: if the table exists raise `ValueError: Table 'Sales' already exists.`
`if_exists="replace"`	drop the old table, create a new one from `df` (old rows AND any UNIQUE rule are lost)
`if_exists="append"`	keep the table, add `df`'s rows at the end (columns must match)
`index=False`	do not store the DataFrame's 0,1,2.. row numbers. Without it you get an extra column called `index`
`dtype={"Price": "REAL"}`	optional: choose a column's SQLite type yourself (plain strings such as `"TEXT"`, `"INTEGER"`, `"REAL"`)

What types SQLite ends up with (checked with `PRAGMA table_info`):

pandas	SQLite column	Notes
`int64`	`INTEGER`	
`float64`	`REAL`	`NaN` becomes `NULL`
text (`str`/`object`)	`TEXT`	
`bool`	`INTEGER`	stored as 1 / 0, read back as 1 / 0 (not True/False)
`datetime64`	`TIMESTAMP`	stored as text `'2023-01-05 00:00:00'` - the time part comes along. **Convert with `.dt.strftime("%Y-%m-%d")` first** so you get clean `'2023-01-05'`. `strftime('%Y', Date)` works either way.

VERIFY (read it back and compare):

```
print(pd.read_sql_query("SELECT * FROM Sales", conn))   # looks right?
n = pd.read_sql_query("SELECT COUNT(*) AS n FROM Sales", conn)["n"][0]
print(df.shape, n)                       # (3, 5) 3  -> same row count
cursor.execute("PRAGMA table_info(Sales)")
for col in cursor.fetchall():            # (cid, name, type, notnull, ...)
    print(col[1], col[2])                # Date TEXT / Quantity INTEGER ...
```

Run result: `df.shape` is `(3, 5)` and the count is `3`. After a second `to_sql(..., if_exists="append")` the count is 6; after `"replace"` it is 3 again.

TRAP: `to_sql` creates **no PRIMARY KEY / UNIQUE**, so it cannot prevent duplicates. Use `drop_duplicates()` first and/or create the table yourself (3.1) then `append`.

8.3 SELECT, WHERE, ORDER BY, LIMIT

Analogy: asking the clerk **"bring me the cards that ..., sorted by ..., just the first few"**. `SELECT` = which boxes of the card to read, `WHERE` = which cards, `ORDER BY` = the sort, `LIMIT` = how many.

WHEN: any "display / list / find" question. Fixed clause order: `SELECT ... FROM ... WHERE ... GROUP BY ... ORDER BY ... LIMIT ...`.

```
def show(sql, params=()):                # tiny helper: run + print rows
    cursor.execute(sql, params)
    for row in cursor.fetchall():
        print("  ", row)

show("SELECT Product, Price FROM Sales")                    # some columns
show("SELECT * FROM Sales WHERE Price > 10 AND Quantity >= 7")
show("SELECT * FROM Sales WHERE Product = 'Water' OR Product = 'Biscuits'")
show("SELECT * FROM Sales WHERE Product IN ('Water', 'Biscuits')")
show("SELECT * FROM Sales WHERE Product NOT IN ('Water', 'Biscuits')")
show("SELECT * FROM Sales WHERE Product LIKE 'C%'")         # starts with C
show("SELECT * FROM Sales WHERE Product LIKE '%ice%'")      # contains ice
show("SELECT * FROM Sales WHERE Quantity BETWEEN 4 AND 7")  # 4 and 7 included
show("""SELECT * FROM Sales
        WHERE Date BETWEEN '2023-01-01' AND '2023-12-31'""")
show("""SELECT * FROM Sales
        WHERE (Product = 'Chocolate' OR Price > 12)
        AND Date >= '2024-01-01'""")                        # brackets!
show("SELECT * FROM Sales ORDER BY Price ASC")              # low -> high
show("SELECT * FROM Sales ORDER BY Product ASC, Total DESC")  # 2 sort keys
show("SELECT Product, Total FROM Sales ORDER BY Total DESC LIMIT 2")
show("SELECT DISTINCT Product FROM Sales")                  # no repeats
```

What every part does:

Piece	Meaning
`=`, `!=`, `<`, `>`, `<=`, `>=`	comparisons (a **single** `=`, not `==`)
`AND` / `OR` / `NOT`	combine tests. `AND` binds tighter than `OR`, so use **brackets** when mixing
`IN ('a','b')`	matches any value in the list (shorter than many ORs)
`LIKE 'C%'`	text pattern: `%` = any characters, `_` = exactly one. Not case-sensitive for plain letters
`BETWEEN a AND b`	`a <= x <= b` - **both ends included**. Works on ISO date text
`ORDER BY x ASC`	smallest first (default); `DESC` biggest first. `ORDER BY a, b DESC` sorts by `a`, ties by `b`
`LIMIT n`	only the first `n` rows (with `ORDER BY ... DESC` = "top n")
`DISTINCT`	remove repeated rows
`expr AS name`	give a computed column a name (`Quantity * Price AS Revenue`)

Run results (5 sample rows): `Price > 10 AND Quantity >= 7` gives the Biscuits and Orange Juice rows; `LIKE 'C%'` gives the two Chocolate rows; `BETWEEN 4 AND 7` gives Quantity 5, 7, 4; `LIMIT 2` gives Biscuits (140.0) then Orange Juice (100.0).

VERIFY: count what you got and compare with what you expect: `len(cursor.fetchall())`, or `SELECT COUNT(*) FROM Sales WHERE ...`.

8.4 UPDATE queries (full worked section)

Analogy: the clerk **crosses out a value on existing cards and writes a new one**. `SET` = what to write, `WHERE` = which cards. *No WHERE = the clerk rewrites EVERY card in the drawer.*

WHEN: "change the price / correct a record / mark as done". Pattern: **look, change, look again, save**.

```
# 1. BEFORE: look at the rows you are about to change
show("SELECT Product, Price FROM Sales WHERE Product = 'Chocolate'")
#    ('Chocolate', 10.0)  ('Chocolate', 10.0)

# 2. UPDATE one column, values passed with ?
cursor.execute("UPDATE Sales SET Price = ? WHERE Product = ?",
               (11.0, "Chocolate"))
print("rows changed:", cursor.rowcount)           # 2  (as expected)
conn.commit()                                     # 3. make it permanent

# 4. AFTER: prove it
show("SELECT Product, Price FROM Sales WHERE Product = 'Chocolate'")
#    ('Chocolate', 11.0)  ('Chocolate', 11.0)
```

What every part does: - `UPDATE Sales` - which table. `SET Price = ?` - which column gets the new value. `WHERE Product = ?` - which rows. The `?` values come from the tuple **in order**: first `?` = `11.0`, second `?` = `"Chocolate"`. - `cursor.rowcount` - how many rows the last UPDATE changed. It is your safety check: expected 2, got 2. - `conn.commit()` - saves. Until then only your connection sees the change.

Update several columns, with more than one condition:

```
cursor.execute("""UPDATE Sales SET Price = ?, Total = Quantity * ?
                  WHERE Product = ? AND Date >= ?""",
               (13.0, 13.0, "Chocolate", "2024-01-01"))
print("rows changed:", cursor.rowcount)           # 1
conn.commit()
show("SELECT * FROM Sales WHERE Product = 'Chocolate'")
#    ('2023-01-05', 'Chocolate', 5, 11.0, 50.0)   <- 2023 row untouched
#    ('2024-03-15', 'Chocolate', 4, 13.0, 52.0)   <- 4 * 13.0 = 52.0
```

Separate the columns with commas after `SET`. Inside `SET` a column name can appear on the right (`Total = Quantity * ?`): the new value is computed from the **current** row. Note the first row still shows `Total 50.0` next to `Price 11.0`: changing `Price` does not change `Total` by itself - update both together.

Update with an arithmetic expression (no Python needed):

```
cursor.execute("UPDATE Sales SET Price = Price * 1.1 "
               "WHERE strftime('%Y', Date) = ?", ("2023",))
print("rows changed:", cursor.rowcount)           # 3  (the 2023 rows)
conn.commit()
show("SELECT Date, Product, ROUND(Price, 2) FROM Sales")
#    ('2023-01-05', 'Chocolate', 12.1)  ('2023-02-10', 'Biscuits', 22.0) ...
```

`Price = Price * 1.1` means "new Price = old Price x 1.1" (10% rise). Run it **once** - each run multiplies again.

DANGER - forgetting WHERE:

```
cursor.execute("UPDATE Sales SET Quantity = Quantity + 1")      # no WHERE!
print(cursor.rowcount)                            # 5 = ALL five rows
conn.rollback()                                   # undo: not committed yet
```

SQL does not ask "are you sure?". The only warning is `rowcount` being the whole table. Habit: write the `WHERE` **first**, run the matching `SELECT ... WHERE ...` to see which rows it hits, then change `SELECT *` into `UPDATE ... SET`.

Nothing matched: a misspelt name gives `rowcount == 0` and no error - test it:

```
cursor.execute("UPDATE Sales SET Price = ? WHERE Product = ?",
               (1.0, "Choclate"))
if cursor.rowcount == 0:
    print("No such product - nothing updated")
```

8.5 Parameterised queries with `?`

Analogy: a **form with blanks**. The SQL sentence is printed on the form once; the values are written into the blanks separately, so a value can never be mistaken for part of the sentence.

WHEN: whenever any part of the SQL comes from a variable or from `input()`.

```
cursor.execute("SELECT * FROM Names WHERE Surname = ?", ("Smith",))
cursor.execute("SELECT * FROM Names WHERE Surname = ? AND FirstName = ?",
               ("Smith", "Jane"))
```

What every part does: - One `?` per value; the second argument is a **tuple** (a list also works) with the values in the same order. - **One value needs a trailing comma:** `("Smith",)`. `("Smith")` is just a string in brackets, and SQLite then sees 5 separate characters: `Incorrect number of bindings supplied. The current statement uses 1, and there are 5 supplied.`

LIKE with `%` - the `%` belongs in the **value**, never next to the `?`:

```
word = "Sm"
cursor.execute("SELECT * FROM Names WHERE Surname LIKE ?", (word + "%",))
cursor.execute("SELECT * FROM Names WHERE Surname LIKE ?", (f"%{word}%",))
# WRONG:  LIKE '%?%'  -> the ? is inside quotes, so it is not a placeholder
```

IN-list - one `?` per item; build the right number of marks:

```
wanted = [1, 3, 4]
marks = ", ".join("?" * len(wanted))              # "?, ?, ?"
cursor.execute(f"SELECT * FROM Names WHERE ID IN ({marks})", wanted)
```

The f-string here only inserts the **question marks** (harmless); the real values still travel through `?`.

Why never f-strings for values (test run):

```
evil = "x' OR '1'='1"
cursor.execute(f"SELECT * FROM Names WHERE Surname = '{evil}'")  # 4 rows!
cursor.execute("SELECT * FROM Names WHERE Surname = ?", (evil,)) # 0 rows
```

The f-string version let the user's text rewrite the query and return every row (SQL injection); a name like `O'Brien` simply crashes it. With `?` the text is treated as plain data. **NOTE:** `?` can only stand for **values**, not for table or column names (`SELECT * FROM ?` is a syntax error). Named style also exists: `WHERE Surname = :s` with `{"s": "Smith"}`.

VERIFY: print `cursor.fetchall()` for a name you know exists, and for one you know does not (`[]`).

8.6 Committing database changes

Analogy: `execute` writes on a **draft form**; `commit()` **stamps and files** it in the cabinet. Close without stamping and the draft is binned.

WHEN: after every INSERT / UPDATE / DELETE (SELECT needs no commit).

```
cursor.execute("INSERT INTO Pets VALUES ('Rex')")
print(conn.in_transaction)              # True  = draft, not yet saved
conn.commit()                           # filed
```

What happens (test run with a **second** connection acting as "another program"):

```
same connection sees:     1      <- your own draft is visible to you
outside sees (no commit): 0      <- nobody else can see it
outside sees (committed): 1
close() without commit:   the row is LOST
```

`with conn:` **(context manager):**

```
with conn:                              # commit at the end if no error
    conn.execute("INSERT INTO Pets VALUES ('Tom')")

try:
    with conn:                          # error inside -> ROLLBACK
        conn.execute("INSERT INTO Pets VALUES ('Ann')")
        conn.execute("INSERT INTO Pets VALUES ('Rex')")   # duplicate!
except sqlite3.IntegrityError as err:
    print("error:", err)                # Ann is rolled back too
```

- `with conn:` = *commit if all went well, rollback if an exception escaped*. It does **not** close the connection - call `conn.close()` yourself. The lab idiom `with sqlite3.connect("x.db") as db:` has the same behaviour.
- `conn.rollback()` throws away uncommitted changes (the "undo").
- Commit once after a loop of inserts (faster than once per row).

VERIFY: reconnect (or use a second connection) and `SELECT COUNT(*)`: the number only changes after the commit.

8.7 Retrieve query results and export them to CSV (3 ways)

Analogy: the clerk brings a **pile of cards** (the result); exporting means **copying the pile onto a ruled sheet** (the CSV) with a heading row.

All three use the same query. Give computed columns an alias so the CSV header is readable:

```
import csv
import sqlite3
import pandas as pd

sql = """SELECT Product, SUM(Quantity) AS TotalQty, SUM(Total) AS Revenue
         FROM Sales WHERE strftime('%Y', Date) = ?
         GROUP BY Product ORDER BY Revenue DESC"""
```

Way 1 - pandas (shortest, best in the exam)

```
result = pd.read_sql_query(sql, conn, params=("2023",))   # -> DataFrame
result.to_csv("out1.csv", index=False)                    # DataFrame -> CSV
```

`params=` takes the same tuple as `execute`. `index=False` keeps the useless 0,1,2 column out of the file.

Way 2 - cursor + `csv` module (no pandas needed)

```
cursor.execute(sql, ("2023",))
rows = cursor.fetchall()                          # list of tuples
headers = [col[0] for col in cursor.description]  # column names
with open("out2.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(headers)                      # heading row ONCE
    writer.writerows(rows)                        # all data rows at once
```

`cursor.description` is a tuple with one entry per result column; the first item of each entry (`col[0]`) is the column name (the alias, e.g. `TotalQty`). It is only filled **after** an `execute`.

Way 3 - DataFrame from the cursor

```
result3 = pd.DataFrame(rows, columns=headers)     # rows + names -> table
result3.to_csv("out3.csv", index=False)
```

VERIFY - read the exported file back:

```
print(pd.read_csv("out1.csv"))
```

```
     Product  TotalQty  Revenue
0   Biscuits         7    140.0
1  Chocolate         8     80.0
```

All three files were identical in the test (compared byte for byte). `out2.csv` as plain text:

```
Product,TotalQty,Revenue
Biscuits,7,140.0
Chocolate,8,80.0
```

TRAP: `result.to_csv("out.csv")` without `index=False` gives a first line `,Product,TotalQty,Revenue` (an unnamed index column). With the csv module, forgetting `newline=""` puts blank lines between rows on Windows.

8.8 Cheat-sheet: the 10 SQL statements to know

#	Purpose	Statement
1	Create a table	`CREATE TABLE IF NOT EXISTS Sales (Date TEXT, Product TEXT, Quantity INTEGER, Price REAL, UNIQUE (Date, Product))`
2	Insert a row	`INSERT INTO Sales VALUES (?, ?, ?, ?)`
3	Insert, skip duplicates	`INSERT OR IGNORE INTO Sales VALUES (?, ?, ?, ?)` (needs UNIQUE/PRIMARY KEY; `rowcount` 0 = skipped)
4	Read + filter	`SELECT Product, Price FROM Sales WHERE Quantity > ?`
5	Sort + top n	`SELECT * FROM Sales ORDER BY Price DESC LIMIT 3`
6	Group + total	`SELECT Product, SUM(Quantity) FROM Sales GROUP BY Product`
7	Change rows	`UPDATE Sales SET Price = ? WHERE Product = ?`
8	Delete rows	`DELETE FROM Sales WHERE Product = ?` (no WHERE = clear all)
9	Remove the table	`DROP TABLE IF EXISTS Sales`
10	Inspect the database	`SELECT name FROM sqlite_master WHERE type = 'table'` and `PRAGMA table_info(Sales)`

Python wrapper for all of them: `cursor.execute(statement, (values,))`, then `conn.commit()` for 1, 2, 3, 7, 8, 9; `cursor.fetchone()` / `cursor.fetchall()` for 4, 5, 6, 10. All ten were run in a test (statement 3 gave `rowcount 0` on a repeat; 7 and 8 each gave `rowcount 1`; after 9 the table list was empty).